

**PROJECT REPORT OF INDUSTRY ORIENTED HANDS ON EXPERIENCE
(IOHE)**

On

AI Career Coach and Resume Optimization

submitted in partial fulfilment of the requirements for the award of degree of

BACHELOR OF ENGINEERING

In

COMPUTER SCIENCE AND ENGINEERING

Submitted by:

Vansh Thakur(2210992499)

Aryan Thakur(2210991391)

Aadil Verma(2210991102)

Harshit Kumar(2210991635)

Supervised By:

Dr. Rajat Takkar



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY
CHITKARA UNIVERSITY, PUNJAB, INDIA**

CANDIDATE'S DECLARATION

I hereby certify that the work which is being presented in the project report entitled **AI Carrier Coach** in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of my own work carried out under the supervision of **Dr. Rajat Takkar**. The matter presented in this project report has not been submitted in any other university/institute for the award of any degree.

Place: Rajpura

Date: 16-05-2026 .

Aadil Verma(2210991102)

Aryan Thakur(2210991391)

Harshit Kumar(2210991635)

Vansh Thakur(2210992499)

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and belief.

Dr. Rajat Takkar

Assistant Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

ACKNOWLEDGEMENT

It is our pleasure to be indebted to various people, who directly or indirectly contributed in the development of this work and who influenced my thinking, behavior and acts during the course of study.

We express our sincere gratitude to all for providing me an opportunity to undergo Integrated Project as a part of the curriculum.

We are thankful to Dr. Rajat Takker for his support, cooperation, and motivation provided to us during the training for constant inspiration, presence and blessings.

We also extend our sincere appreciation to Dr. Rajat Takker who provided his valuable suggestions and precious time in accomplishing our integrated project report.

Lastly, we would like to thank the almighty and our parents for their moral support and friends with whom we shared our day-to-day experience and received lots of suggestions that improve our quality of work.

Aadil Verma
(2210991102)

Aryan Thakur
(2210991391)

Vansh Thakur
(2210992499)

Harshit Kumar
(2210991635)

Table of Contents

S.No.	Title	Page.No
1	Abstract	5
2	Introduction	6
3	Software and Hardware Requirement	7
4	System Design	8-11
5	Technology Stack	12
6	Database Analyzing, design and implementation	13-18
7	Program's Structure Analyzing and GUI Constructing	19-40
8	Testing & Validation	41
9	Security & Performance	42
10	Limitation	43
11	Challenges Faced	44
12	Future Scope	45
13	Conclusion	46
14	References	47

1. Abstract/Keywords

This project report presents the development of an **AI Career Coach** application designed to provide intelligent, personalized career guidance using modern web technologies and AI integration. Developed with **Next.js**, **Express.js**, and **PostgreSQL**, the platform helps users discover career paths, create optimized resumes, practice for interviews, and stay updated with industry trends. Leveraging **Gemini APIs**, the system offers smart recommendations for job roles based on user skills, goals, and preferences. Key features include AI-assisted resume and cover letter generation, mock interview simulations, personalized learning paths, and a user-friendly dashboard. The backend is secured using JWT-based authentication, with Prisma ORM handling database operations and Vercel ensuring seamless cloud deployment. This application aims to bridge the gap between career planning and AI-driven assistance for users navigating today's competitive job market.

Keywords

AI Career Coach, Next.js, Node.js, Express.js, PostgreSQL, Prisma ORM, Gemini API, Career Guidance, Resume Builder, Interview Preparation, JWT Authentication, Personalized Learning, Job Market Insights, AI-Powered Recommendations, Vercel Hosting.

2. INTRODUCTION

2.1 Background

In today's rapidly evolving job market, individuals face increasing challenges in navigating career paths, building strong professional profiles, and preparing effectively for employment opportunities. Traditional career counseling methods often lack personalization, accessibility, and real-time relevance. With the rise of artificial intelligence and web-based platforms, there is a growing demand for intelligent career guidance tools that can adapt to individual goals and market trends.

This project introduces a feature-rich **AI Career Coach** application developed using **Next.js**, **Express.js**, and **PostgreSQL**, designed to bridge the gap between job seekers and career advancement resources. The platform supports role-based access for users and administrators, ensuring secure interaction and personalized services. Users can receive AI-driven career recommendations, generate ATS-friendly resumes and cover letters, prepare for interviews, and monitor their career development through an interactive dashboard. The system integrates **Gemini APIs** for smart analysis, **JWT-based authentication** for security, and is deployed on **Vercel** for reliable cloud access—making career planning more intelligent, accessible, and future-ready.

2.2 Problem Statement

Traditional career counseling methods often lack accessibility, personalization, and adaptability to the dynamic nature of today's job market. Many individuals face difficulties in identifying suitable career paths, building effective resumes, and preparing for interviews without real-time feedback or tailored guidance. These limitations can lead to confusion, underemployment, or missed opportunities—especially for students, recent graduates, and career switchers.

The key problems addressed by this project are:

- Lack of personalized career recommendations based on individual goals and skillsets
- Difficulty in creating professional resumes and cover letters that meet current industry standards
- Limited access to realistic interview preparation and performance feedback
- Inability to track personal growth and adapt to evolving job market trends.

This project aims to solve these challenges by delivering an AI-powered web application that provides smart career recommendations, resume generation, mock interview simulations, and career progress tracking. By integrating AI and user-friendly design, the platform empowers individuals to make informed and confident career decisions.

3. Software and Hardware Requirement Specification

3.1 Methods

The AI Career Coach platform is built using **Next.js** for the frontend and **Node.js/Express.js** for the backend. **PostgreSQL**, managed via **Prisma ORM**, handles secure data storage. The system integrates **Gemini APIs** for AI-powered features like resume analysis and career suggestions. **JWT** ensures secure authentication, and the app follows a **RESTful API** design for smooth frontend-backend communication.

3.2 Programming/Working Environment

The **AI Career Coach** project is developed using modern web technologies. **Next.js** is utilized for building the interactive user interface, while **Node.js** and **Express.js** manage backend logic, API routes, and database interaction. **PostgreSQL** serves as the database, with **Prisma ORM** for efficient schema design and querying. Development is carried out in **Visual Studio Code** as the primary Integrated Development Environment (IDE), and **Postman** is used for testing API endpoints. **Git** and **GitHub** are employed for version control, ensuring smooth collaboration and code management. Additionally, **npm** handles project dependencies, with packages such as **JWT** for secure authentication and **Gemini APIs** integrated for AI-driven career guidance features.

3.3 Requirements to run the application

To run the **AI Career Coach** application, both software and hardware requirements must be met. Software requirements include **Node.js** for running the backend server, **PostgreSQL** for database management, and **Next.js** for the frontend interface. A modern browser such as Google Chrome or Mozilla Firefox is required for accessing the application. Development tools like **Visual Studio Code** and **Postman** are essential for coding and testing. For hardware, a system with at least 4GB of RAM and a dual-core processor is recommended for optimal performance. A stable internet connection is also necessary for cloud database access and application deployment.



4. System Design

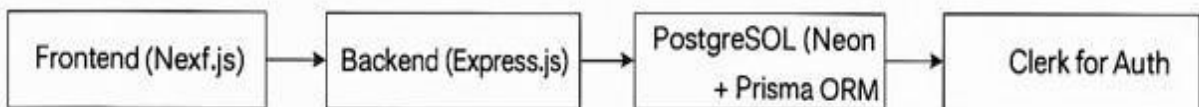
The system design outlines the complete technical structure of the AI Career Coach, detailing how components interact and are managed across the stack.

4.1 Architecture

The application architecture follows a modular full-stack approach:

- Frontend: Developed using Next.js with Chakra UI and shadcn/ui for responsive UI components
- Backend: Built on Node.js and Express.js, handling routing, authentication, and logic
- Database: PostgreSQL hosted on Supabase, managed using Prisma ORM
- Authentication: Handled securely using Clerk and JWT tokens
- AI Integration: Gemini APIs are used for career suggestions, resume analysis, and interview prep

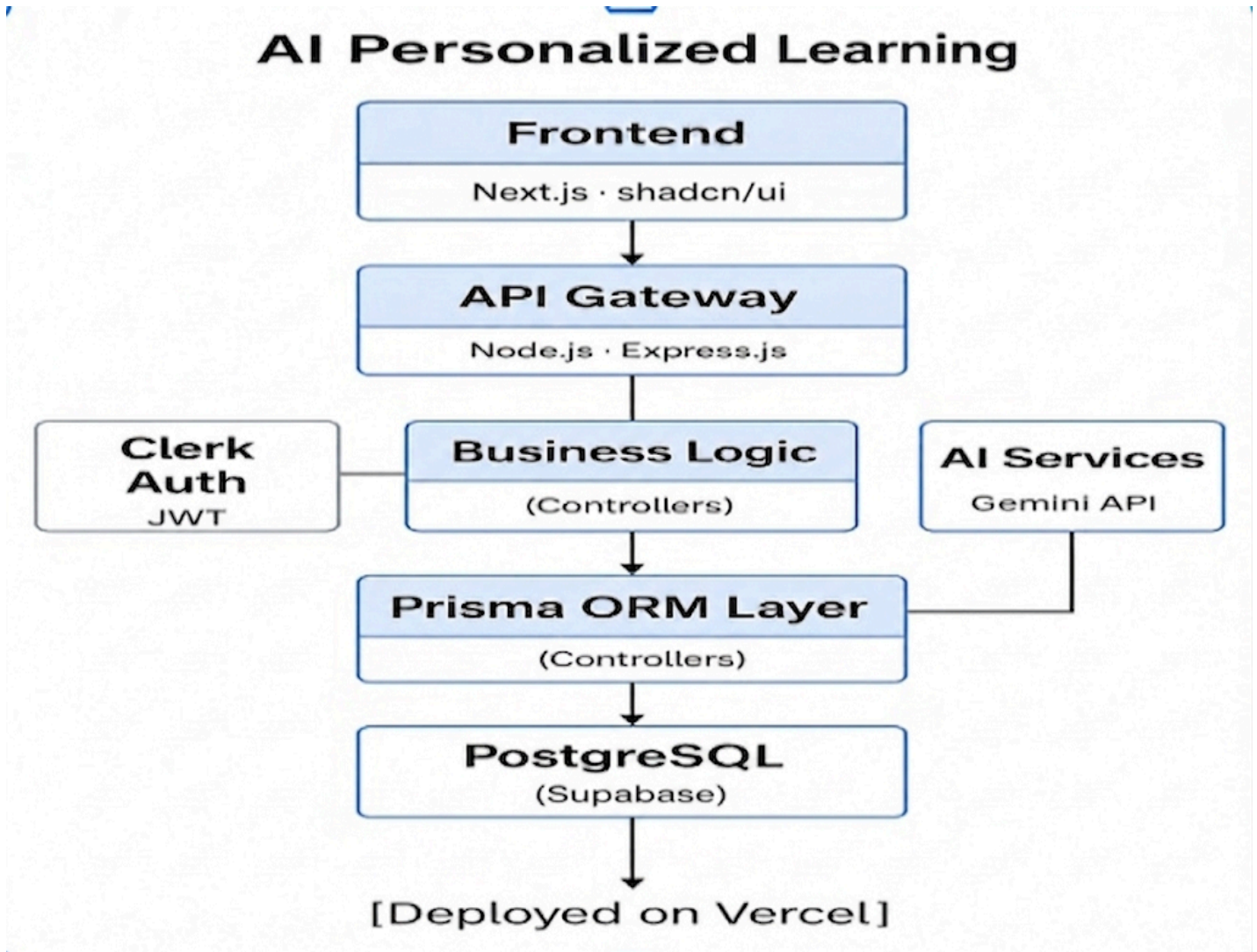
5 Architecture



4.2 Flow:

User → Next.js → Express API → PostgreSQL (via Prisma) → Gemini API (AI) / Clerk (Auth)

4.21 High-Level Design Diagram



4.3 ER Diagram

User

- Attributes: `user_id`, `clerkUserId`, `name`, `email`, `bio`, `industry`, `timestamps`
- Relationships:
 - One user → Many Resumes
 - One user → Many Interviews
 - One user → Many Assessments

Resume

- Attributes: `resume_id`, `user_id` (FK), `skills`, `experience`, `education`, `filePath`

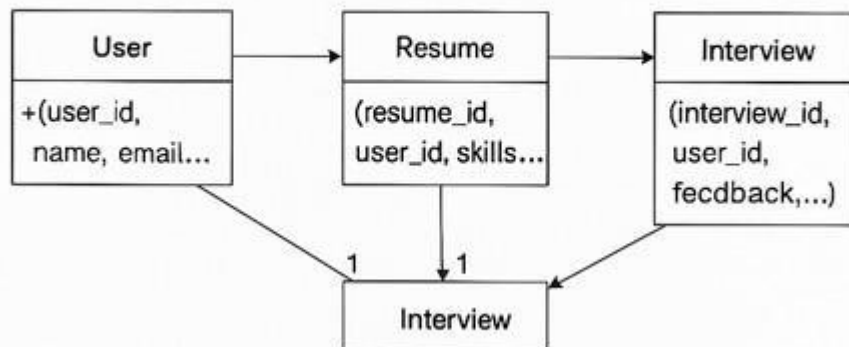
Interview

- Attributes: `interview_id`, `user_id` (FK), `questions` (JSON), `score`, `feedback`, `date`

Assessment

- Attributes: `assessment_id`, `user_id` (FK), `category`, `quizScore`, `improvementTips`

5 ER Diagram



4.4 UML

Diagrams Class

Diagram

- **User:** userId, name, email, password, industry
- **Resume:** resumeId, userId, skills, experience
- **Job:** jobId, title, description, company
- **Interview:** interviewId, userId, score, feedback

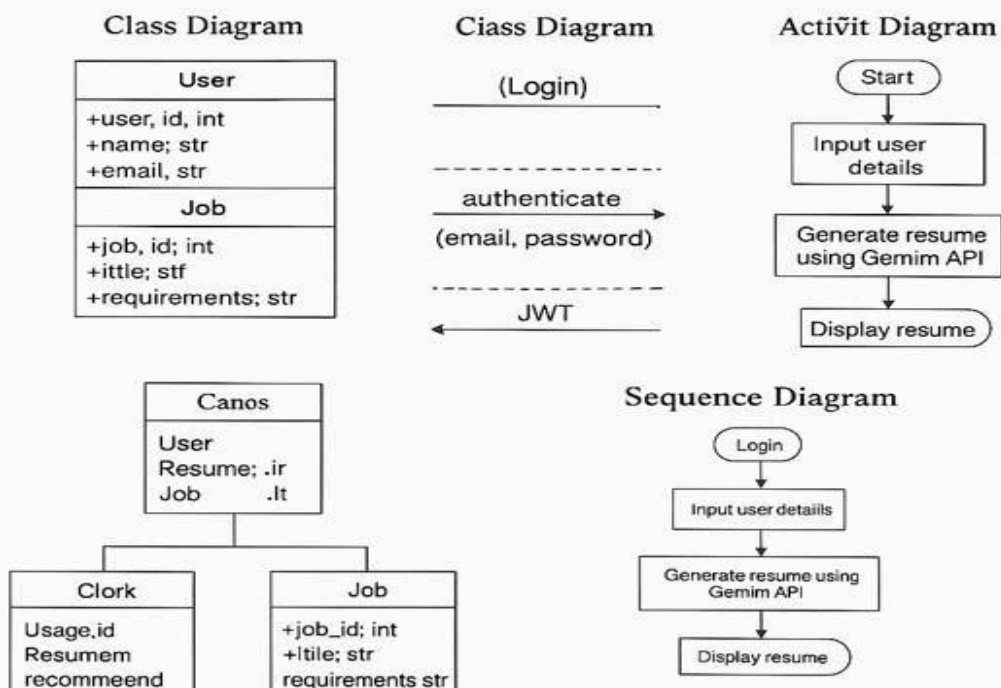
Sequence Diagram – Login Flow

1. User opens login page
2. Sends credentials to `/api/auth/login`
3. Server validates and returns JWT + Clerk session
4. User redirected to dashboard with session token

Activity Diagram – Resume Generation Flow

1. User uploads/responses input
2. Data sent to backend
3. Gemini API processes request
4. Response saved in database
5. Resume preview/downloaded

5 UML Diagrams



5. Technology Stack

◆ Frontend:

Built using **Next.js** for server-side rendering and routing, combined with **Material UI** for a modern, responsive component library. Core logic is implemented in **JavaScript**, enabling a dynamic and user-friendly interface.

◆ Backend:

Developed with **Node.js** and **Express.js**, providing a fast and scalable server-side environment to handle API routes, user requests, and data processing.

◆ Authentication & Security:

Implemented using **JWT (JSON Web Tokens)** to manage secure user authentication and protect API routes, ensuring data privacy and safe session handling.

◆ ORM & Database Management:

Utilized **Prisma ORM** for efficient database schema design, migrations, and simplified database queries within the backend.

◆ AI Integration:

Integrated with **Gemini APIs** to power AI-driven features such as **interview feedback**, **resume analysis**, and **industry insights**, enhancing personalization and user experience.

◆ Testing & Monitoring:

Used **Postman** for thorough API testing and validation of middleware logic during development and debugging phases.

6. Database Analyzing, design and implementation

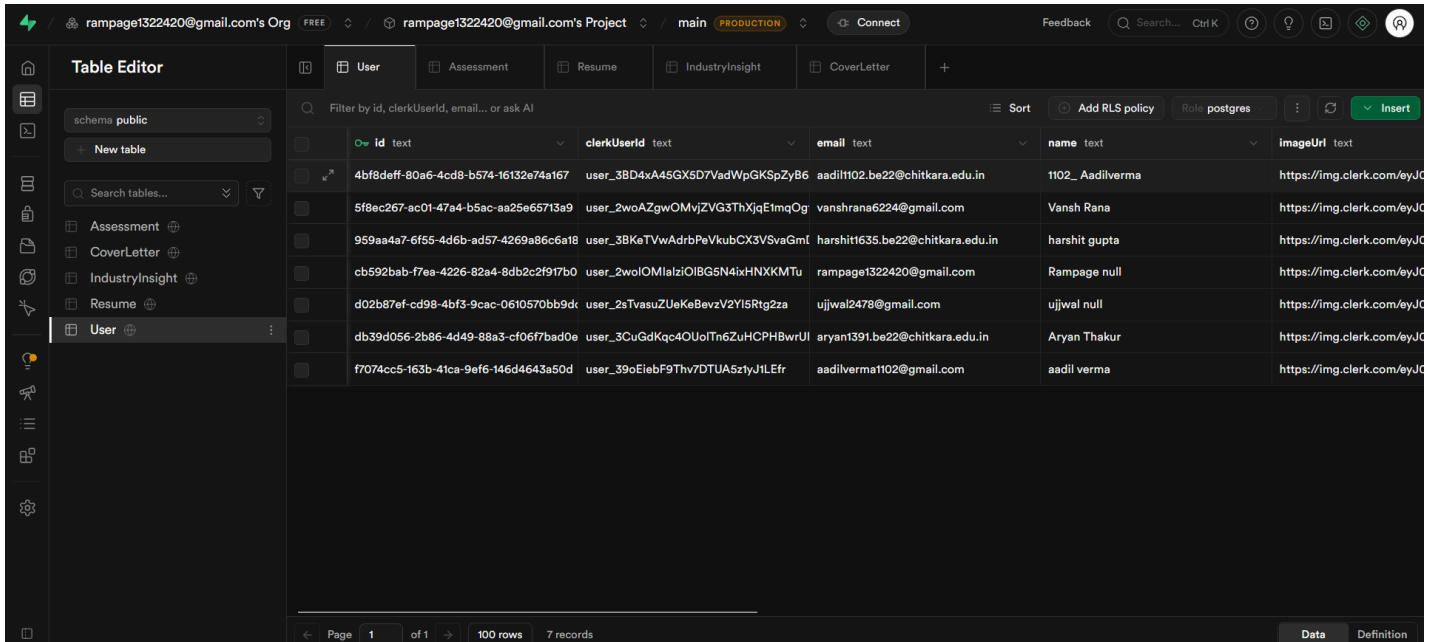
The database for the **AI Career Coach** application is designed using **PostgreSQL**, a relational database known for its robustness and reliability. The core entities in the system include **User**, **Resume**, **Job**, and **Interview**, each represented by separate tables.

- The **User Table** stores information related to the users, including user IDs, personal details, login credentials, and preferences.
- The **Resume Table** holds data regarding the resumes uploaded by users, including the resume ID, user ID, and various resume details (e.g., skills, experience, and education).
- The **Job Table** captures job listings, including company details, job role, required qualifications, and application status.
- The **Interview Table** stores interview results, including mock interview history, user performance, and feedback.

Prisma ORM is used for efficient database management, ensuring seamless communication between the frontend and backend. The relational structure of PostgreSQL allows for structured data storage and efficient querying, ensuring smooth performance and scalability as the system grows. This design provides flexibility for future expansions, such as integrating new features like job recommendations and personalized career insights.

6.1. Database Architecture & Table Snapshots

6.1.1 User Data Management View from Production Database



id	clerkUserId	email	name	imageUrl
4bf8deff-80a6-4cd8-b574-16132e74a167	user_3BD4xA45GX5D7VadWpGKSpZyB6	aadilt102.be22@chitkara.edu.in	1102_ Aadilverma	https://img.clerk.com/eyJ...
5f8ec267-ac01-47a4-b5ac-as25e65713a9	user_2woAZgwOMvjZVG3ThXiqEImqOg	vanshrana6224@gmail.com	Vansh Rana	https://img.clerk.com/eyJ...
959aa4a7-6f55-4d6b-ad57-4269a86c6a18	user_3BKeTVwAdrbPeVkubCX3VSvaGmI	harshit1635.be22@chitkara.edu.in	harshit gupta	https://img.clerk.com/eyJ...
cb592bab-f7ea-4226-82a4-8db2c2f917b0	user_2woOMlalziOIBGSN4ixHNXKMTu	rampage1322420@gmail.com	Rampage null	https://img.clerk.com/eyJ...
d02b87ef-cd98-4bf3-9cac-0610570bb9dc	user_2sTvasuZUeKeBevzV2Y15Rtg2za	ujjwal2478@gmail.com	ujjwal null	https://img.clerk.com/eyJ...
db39d056-2b86-4d49-88a3-cf06f7bad0e	user_3CuGdKqC4OUoITn6ZuHCPHBwrUI	aryant1391.be22@chitkara.edu.in	Aryan Thakur	https://img.clerk.com/eyJ...
f7074cc5-163b-41ca-9ef6-146d4643a50d	user_39oEiebf9ThvDTUASzlyJLEfr	aadilverma1102@gmail.com	aadil verma	https://img.clerk.com/eyJ...

The screenshot above shows the **User** table from the production database of the AI Career Coach platform, accessed via the supabase dashboard under the **elevateAI** project. This table stores registered user information and acts as the foundation for user authentication and personalization.

Each record includes:

- User ID – a unique identifier for each user
- Authentication details – pulled from Clerk integration
- Email and user-specific metadata – to support profile management
- Timestamps – for account creation and last updates

This table ensures seamless user onboarding, secure login, and supports features like personalized dashboards, resume data linking, and tracking user activity throughout the platform.

6.1.2 Assessment Results View from Production Database

id	userid	quizScore	questions	category
cmmvu7ee0004fdlwd07ggup9	cb592bab-f7ea-4226-82a4-8db2c...	50	[{"answer":"Request","question":"In a Spr...	Technical
cmmymnte0001f5043ghsxcom	5f8ec267-ac01-47a4-b5ac-aa25e6...	70	[{"answer":"Regularization","question":"W...	Technical
cmn6toz90001kw045vo08pbp	cb592bab-f7ea-4226-82a4-8db2c...	10	[{"answer":"To prevent instruction record...	Technical
cmnilrnw90001ju04n8j5jdm0	cb592bab-f7ea-4226-82a4-8db2c...	20	[{"answer":"B) Relying on the 'spring.fact...	Technical
cmoyl9akj0001ij04lbu9ux4x	cb592bab-f7ea-4226-82a4-8db2c...	20	[{"answer":"It ensures visibility of writes t...	Technical
cmp2l29qa0001js04oxvtns2j	5f8ec267-ac01-47a4-b5ac-aa25e6...	50	[{"answer":"df.loc[]","question":"Which Pa...	Technical
cmp5a7xfv0003jv04d2fs6lh7	db39d056-2b86-4d49-88a3-cf06...	90	[{"answer":"df.fillna()","question":"Which I...	Technical

This figure displays the **Assessment** table, which logs mock interview and quiz data submitted by users. It forms the core of the Interview Preparation and performance tracking features of the AI Career Coach platform.

Each entry captures:

- User ID – linking each assessment to a specific user
- Score or performance metrics – based on quiz results
- Created and updated timestamps – for tracking attempts over time
- Additional fields – such as quiz type, category, and improvement tips

This structured assessment data enables real-time feedback, performance visualization, and personalized insights, helping users continuously improve their interview readiness.

6.1.3 Cover Letter Results View from Production Database

id	userid	content	jobDescription	companyName
cmmvu3wea0002fdlwbmoaptbl	cb592bab-f7ea-4226-82a4-8db2c...	""markdown [Your Name] [Your Address] i have 2 years of building software in spirir helix		
cmnzjlvsj0001jr04pjs587j7	5f8ec267-ac01-47a4-b5ac-aa25e6...	Here's a professional cover letter for the / i am good at building rag pipelines and ag nebero		
cmp59z1460001jv04h6xfy8es	db39d056-2b86-4d49-88a3-cf06...	[Your Name] [Your Street Address] [Your C SDE at Google		Google

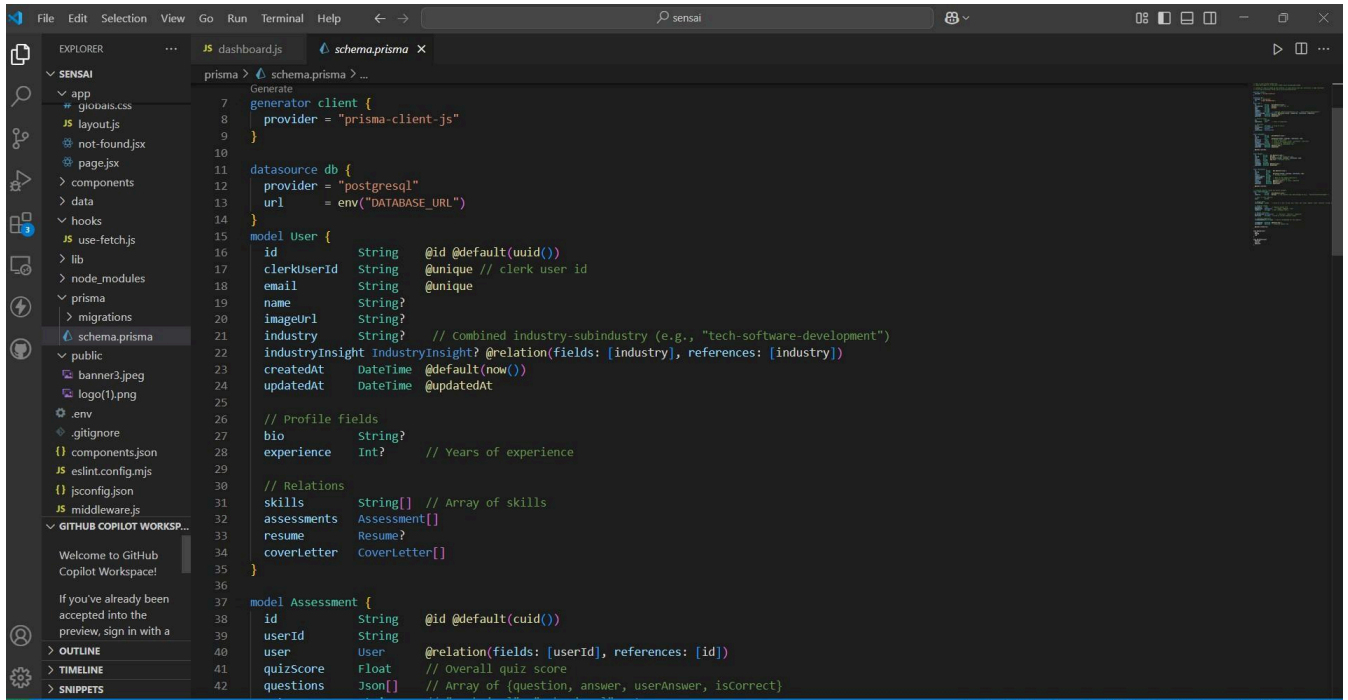
The screenshot above displays the **CoverLetter** table from the production database, accessed via the Supabase dashboard under the **ElevateAI** project. This table stores all AI-generated cover letters created by users on the AI Career Coach platform.

Each record includes:

- User ID – to link the cover letter to a specific user
- Content – the generated cover letter text
- Job Description – user-provided input for context
- Company Name and Job Title – the role targeted by the letter
- Status – indicates whether the letter generation is completed
- Timestamps – **createdAt** and **updatedAt** track when each entry was made or modified

This schema allows users to create, view, and manage their cover letters smoothly through the application. It also reflects the strong connection between the frontend form components and backend data storage, ensuring a reliable and consistent user experience.

6.2.1 Prisma Schema: User and Assessment Models for AI Career Coach Platform



The image shows a Prisma schema used in the AI Career Coach platform. It defines the structure of the database with two main models: **User** and **Assessment**.

- The **User** model stores user data like ID, Clerk auth ID, email, name, image, industry, bio, experience, and timestamps. It also has relations for skills, resume, cover letter, and assessments.
- The **Assessment** model links to the user and contains quiz data like score and questions (stored as JSON).

This schema connects to a PostgreSQL database and uses Clerk for secure user authentication. It helps manage user profiles and assessments in a structured way.



6.2.2 Resume, Cover Letter & Industry Insight Models in Prisma Schema

```
model Resume {
  id          String      @id @default(cuid())
  userId      String      @unique // One resume per user
  user        User        @relation(fields: [userId], references: [id])
  content      String      @db.Text // Markdown content
  ataScore    Float?
  feedback    String?
  createdAt   DateTime     @default(now())
  updatedAt   DateTime     @updatedAt
}

model CoverLetter {
  id          String      @id @default(cuid())
  userId      String      @unique
  user        User        @relation(fields: [userId], references: [id])
  content      String      // Markdown content
  jobDescription String?
  companyName  String      // Name of the company applying to
  jobTitle     String      // Position applying for
  status       String      @default("draft") // draft, completed
  createdAt    DateTime     @default(now())
  updatedAt    DateTime     @updatedAt
}

@@index([userId])

// Combined Industry Trends and Salary Insights
model IndustryInsight {
  id          String      @id @default(cuid())
  industry     String      @unique // The industry this data belongs to (e.g., "tech-software-development")

  // Users in this industry
  users        User[]

  // Salary data
  salaryRanges json[] // Array of { role: string, min: float, max: float, median: float, location: string? }

  // Industry trends
  growthRate    Float      // Industry growth rate
  demandLevel   DemandLevel // "High", "Medium", "Low"
  topSkills     String[]   // Most in-demand skills

  // Market conditions
  marketOutlook MarketOutlook // "Positive", "Neutral", "Negative"
  keyTrends     String[]   // Array of current industry trends

  // Learning suggestions
  recommendedSkills String[] // Skills recommended for the industry

  lastUpdated   DateTime     @default(now())
  nextUpdate    DateTime     // Scheduled update time
}
```

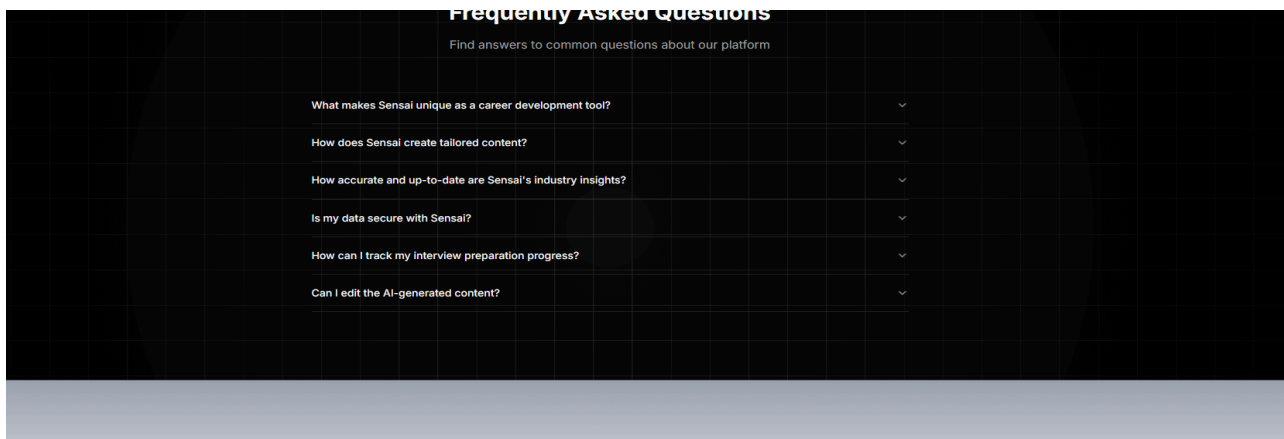
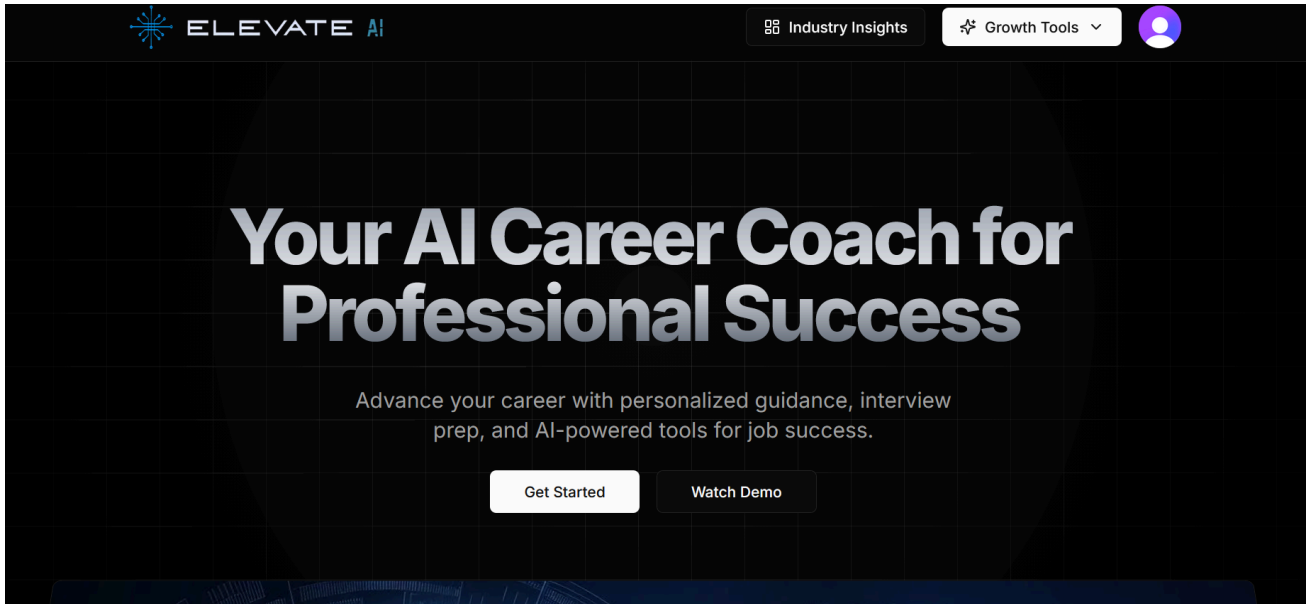
This schema section includes three models:

- **Resume Model:** Stores user resumes, including content, score, feedback, and creation/update timestamps.
- **CoverLetter Model:** Contains cover letter details like company name, job title, content, and status (draft/completed).
- **IndustryInsight Model:** Combines industry trends and salary insights. It stores growth rate, demand level, salary ranges, trending skills, and recommendations for users in that industry.

These models help in managing personalized career insights and document generation for users within the AI Career Coach platform.

7. Program's Structure Analyzing and GUI Constructing

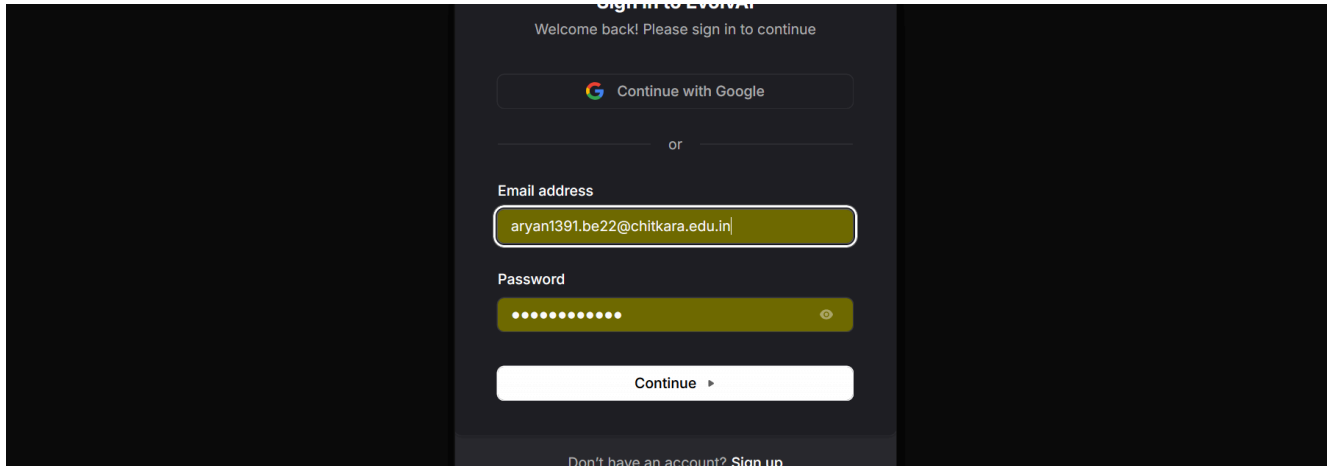
7.1 - Home Page



The AI Career Coach landing page is a modern, responsive web interface built with Next.js and ShadCN UI. It showcases features like AI-powered resume building, interview preparation, and personalized career guidance. With sections including testimonials, FAQs, and interactive elements, it aims to engage users and drive them toward exploring the platform. Integrated with icons, cards, and animations, the design emphasizes usability, professionalism, and AI-driven support for career advancement.

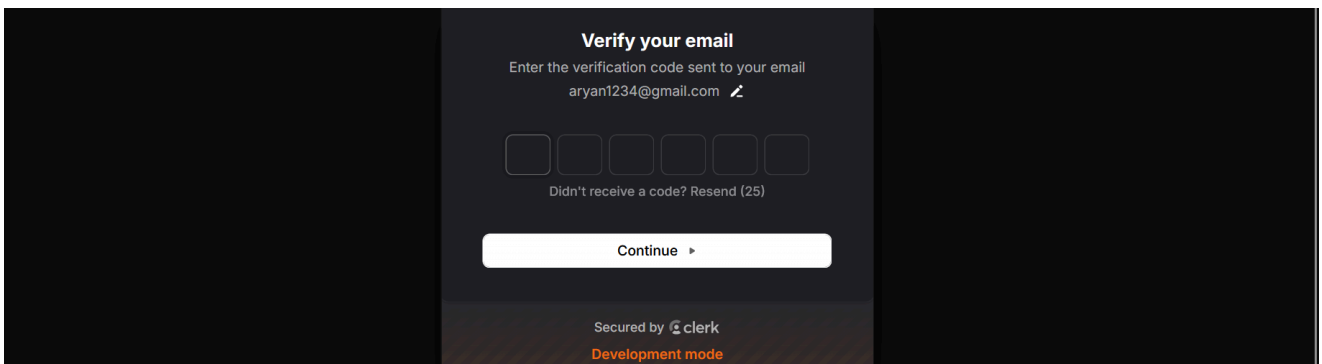


7.2 Login Page



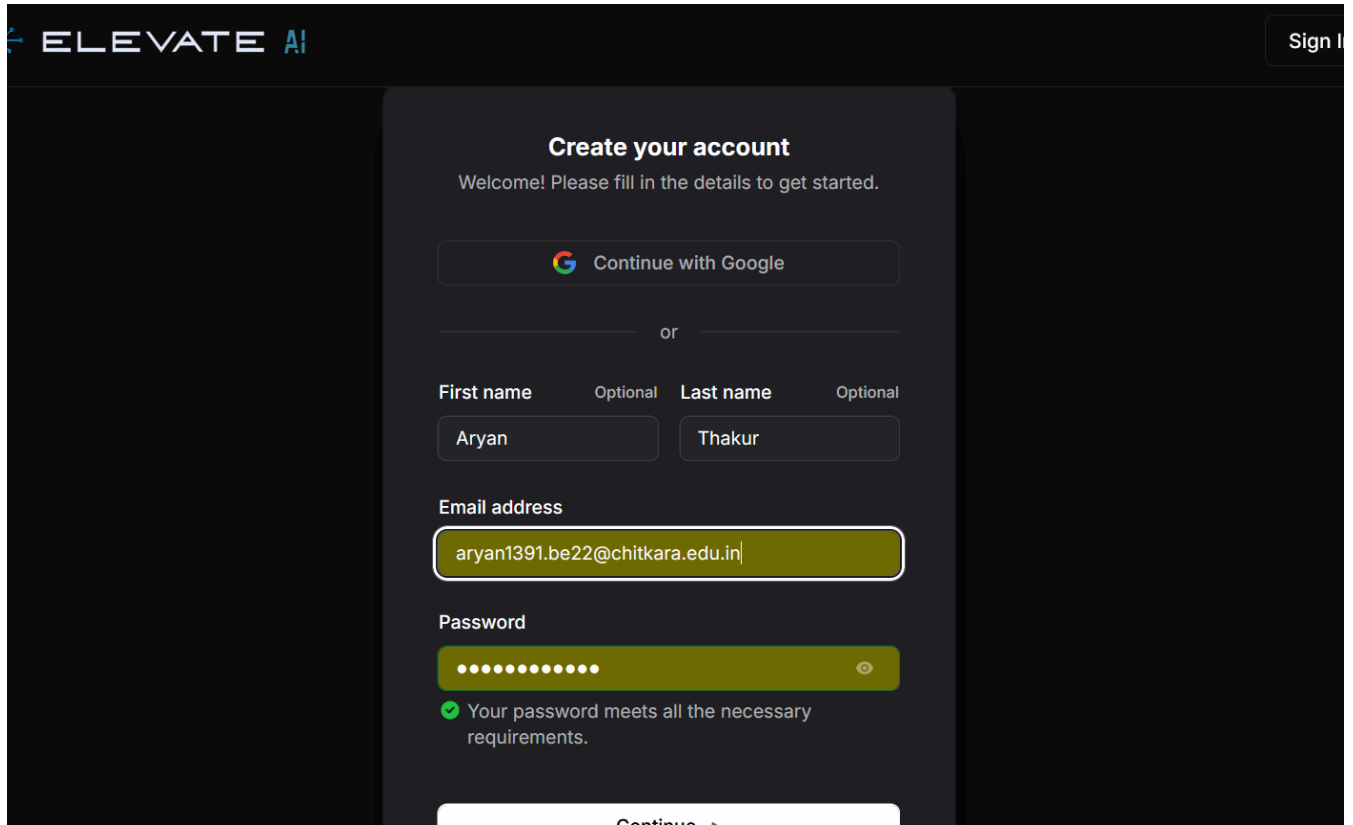
This page implements user authentication using Clerk's [SignIn](#) component, seamlessly integrating secure login functionality into the AI Career Coach platform. Leveraging Clerk's pre-built UI and authentication flow, it simplifies the sign-in process while ensuring robust security. The component is embedded within a Next.js page, offering a clean and responsive user experience, and serves as the gateway for users to access personalized career tools and dashboards.

7.3 Two Factor Authentication



The AI Career Coach platform enhances account security by integrating two-step verification (2FA) through Clerk. This feature adds an additional layer of protection during sign-in, requiring users to verify their identity via a secondary method, such as a one-time password (OTP) sent to their email or phone. By incorporating Clerk's built-in 2FA capabilities, the platform ensures secure access to user data and maintains trust in its AI-powered career guidance services.

7.4 User SignUp



ELEVATE AI

Sign In

Create your account

Welcome! Please fill in the details to get started.

 Continue with Google

or

First name Optional Last name Optional

Aryan Thakur

Email address

aryan1391.be22@chitkara.edu.in

Password

••••••••••

✓ Your password meets all the necessary requirements.

Continue

This page integrates **Clerk's SignUp component** within a Next.js route to provide a seamless and secure registration experience for users. The AI Career Coach platform leverages Clerk's **pre-built authentication UI** to handle user onboarding efficiently and professionally.

New users can sign up using traditional **email and password**, or via **OAuth-based social logins** such as Google, GitHub, and other supported identity providers. OAuth integration enables users to authenticate with their existing accounts, reducing friction during registration and enhancing overall user experience.

Clerk abstracts the complexity of OAuth workflows, handling **token exchanges, redirects, and identity verification** securely. Upon successful login via OAuth, Clerk issues a secure session token, which is then validated and stored using **JWT-based authentication** on the backend. This not only ensures data protection and compliance with modern security standards but also supports **role-based access control** within the platform.

7.5 User OnBoarding Page

ELEVATE AI Industry Insights Growth Tools

Complete Your Profile

Select your industry to get personalized career insights and recommendations.

Industry

Technology

Specialization

Artificial Intelligence/Machine Learning

Years of Experience

1

Skills

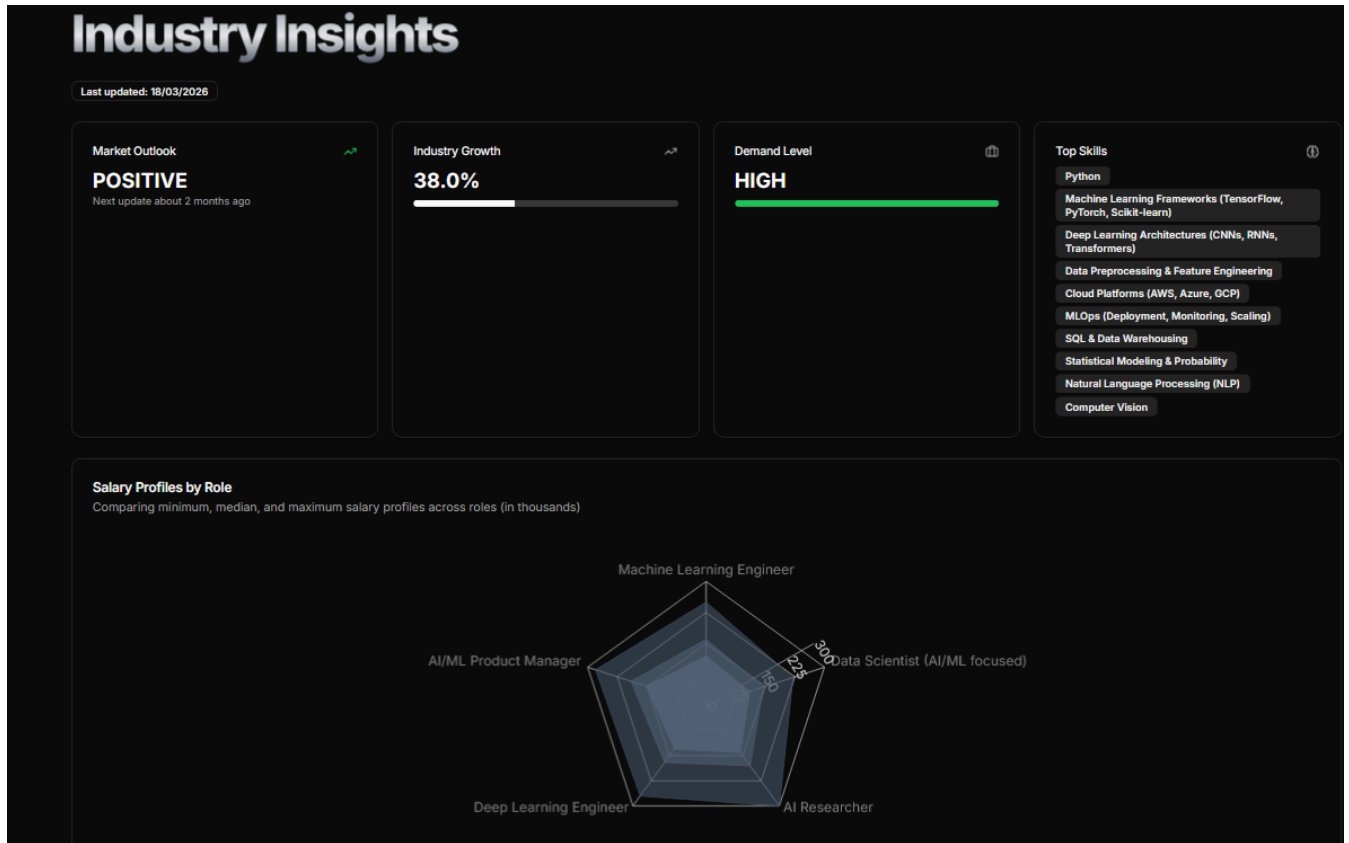
Python

Separate multiple skills with commas

The Onboarding Form helps users customize their **AI Career Coach** experience by collecting important professional details such as industry, specialization, experience, key skills, and a short bio. It is built using **React Hook Form** with **Zod validation** to ensure correct data input and real-time error handling.

The form updates dynamically—for example, choosing an industry shows relevant specializations—making the process **smooth and user-friendly**. Once submitted, the data is securely sent to the backend, the profile is updated, and a **toast notification** confirms **success**. After that, users are redirected to their dashboard to begin exploring personalized **career features**.

7.6 Industry Insights – Career Insights at a Glance



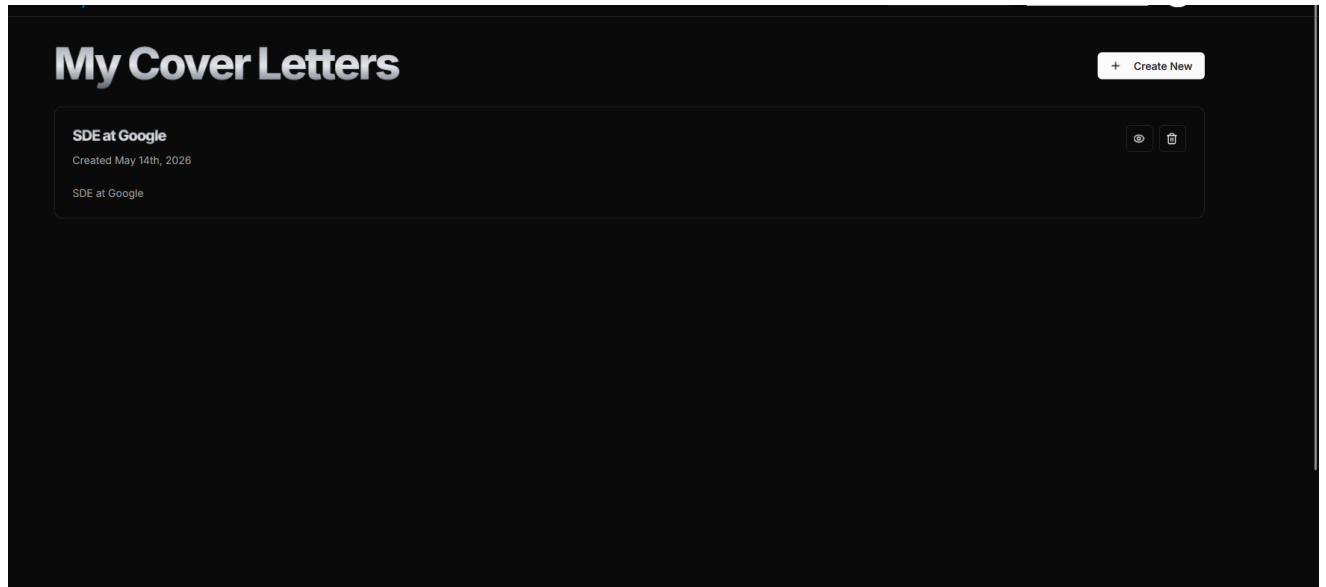
The **DashboardView** component offers users a clear and engaging overview of their career path through interactive visualizations and key industry data. It displays salary ranges, industry growth, market trends, and skill demand using clean UI cards and charts built with the **Recharts** library.

Dynamic bar charts with tooltips and labels make data easy to understand, while **date-fns** is used to show human-readable dates for updates. Visual elements like progress bars and badges quickly convey important metrics such as market sentiment and growth rates.

By turning complex data into simple insights, the dashboard helps users make informed decisions, stay aligned with job market trends, and plan their professional growth effectively.



7.7 AI Cover-Letter

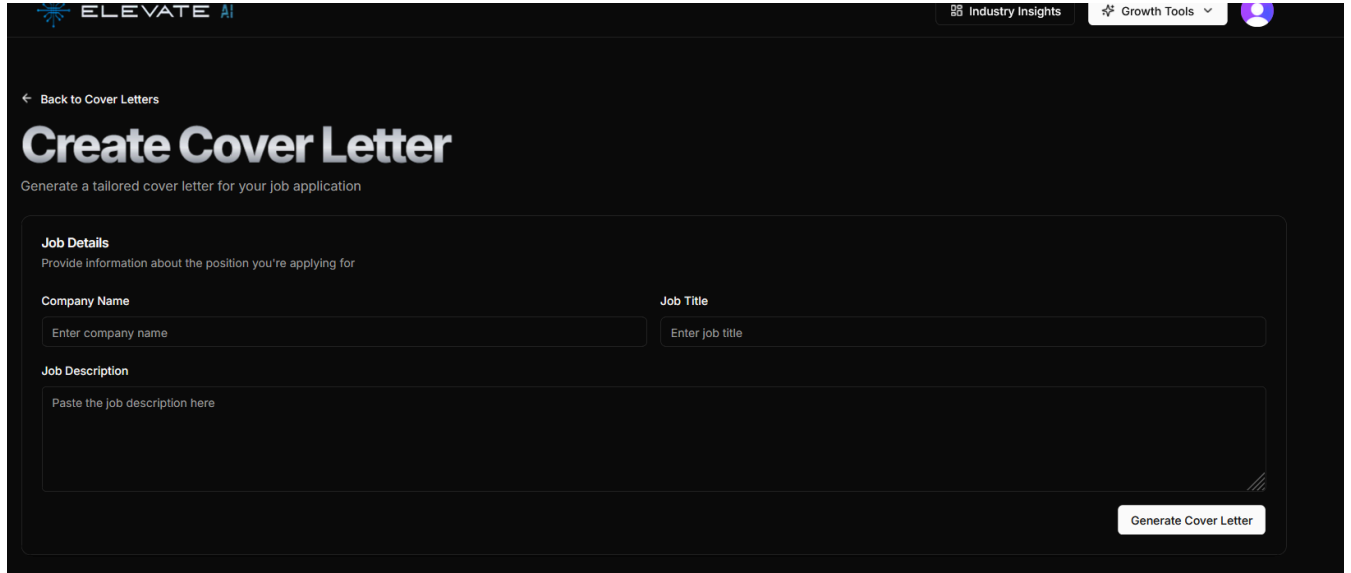


The **CoverLetterPage** component provides a clean and responsive interface for users to view and manage their cover letters. It fetches user data through the **getcover letters** action to ensure up-to-date entries.

A prominent header titled “**My Cover Letters**” is paired with a “**Create New**” button featuring an icon, allowing users to easily add new letters. The layout adapts across screen sizes using **flexbox**, keeping it user-friendly on all devices.

Fetches data is passed to the **CoverLetterList** component, which displays each cover letter in a structured list. This modular setup improves performance, readability, and simplifies user interaction with cover letter management.

7.8 Improve with AI – Smart Personalization for Cover Letters



The screenshot shows a web interface for 'ELEVATE AI'. At the top, there's a navigation bar with 'Industry Insights' and 'Growth Tools'. Below this, a 'Back to Cover Letters' link is visible. The main heading is 'Create Cover Letter', followed by the subtitle 'Generate a tailored cover letter for your job application'. The form is divided into sections: 'Job Details' with a prompt 'Provide information about the position you're applying for', and 'Job Description' with a prompt 'Paste the job description here'. Under 'Job Details', there are two input fields: 'Company Name' (with placeholder 'Enter company name') and 'Job Title' (with placeholder 'Enter job title'). A 'Generate Cover Letter' button is located at the bottom right of the form.

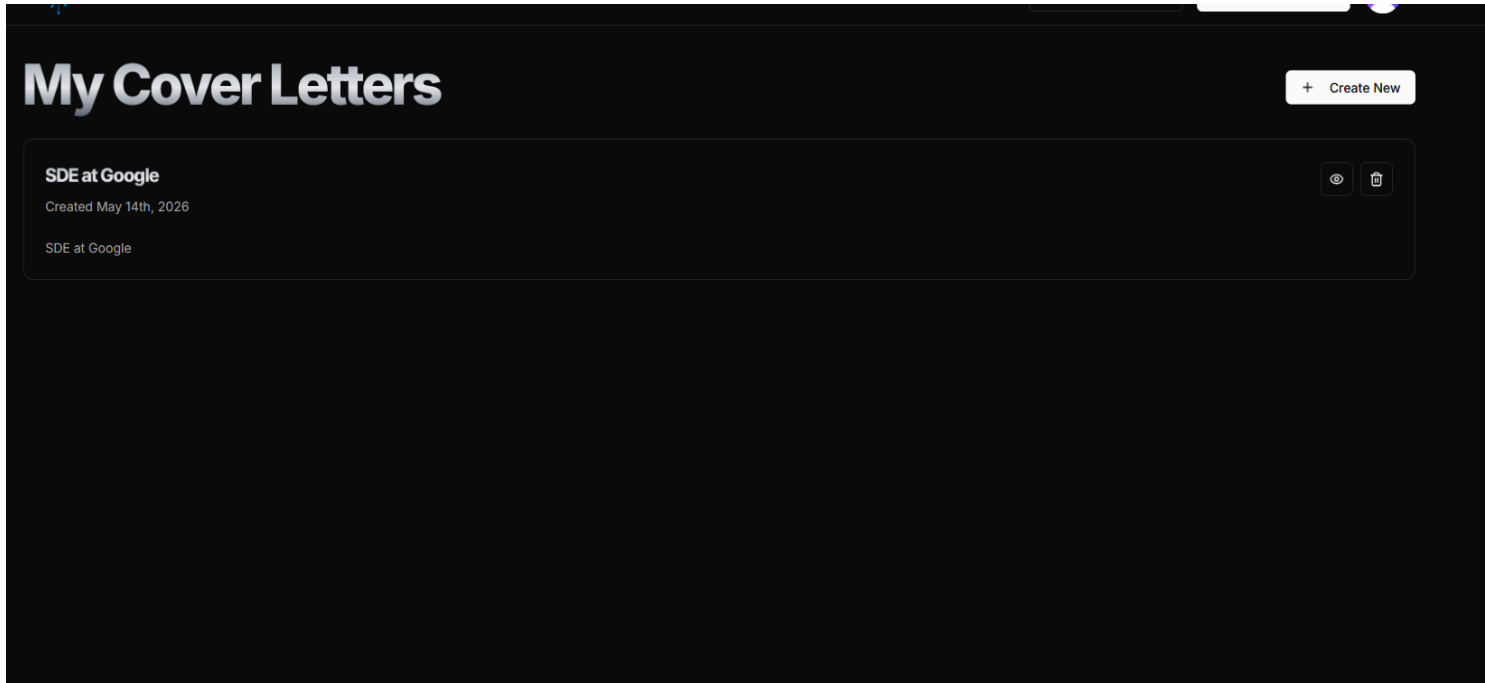
The **CoverLetterGenerator** component offers a smooth and user-friendly interface for generating AI-powered cover letters. Built using **React Hook Form** with **Zod validation**, it captures key inputs like company name, job title, and job description, while providing real-time error feedback.

On form submission, an asynchronous request is sent to generate the cover letter. During this process, the submit button is disabled and shows a loading spinner for improved UX. After successful generation, a **toast notification** confirms completion, the form resets, and the user is redirected to view the new letter.

This component ensures efficient input handling, smooth navigation, and intuitive interaction. It also provides a scalable foundation for future enhancements like draft-saving or additional personalization fields.



7.9 My Cover Letters-Your Personalized Repository



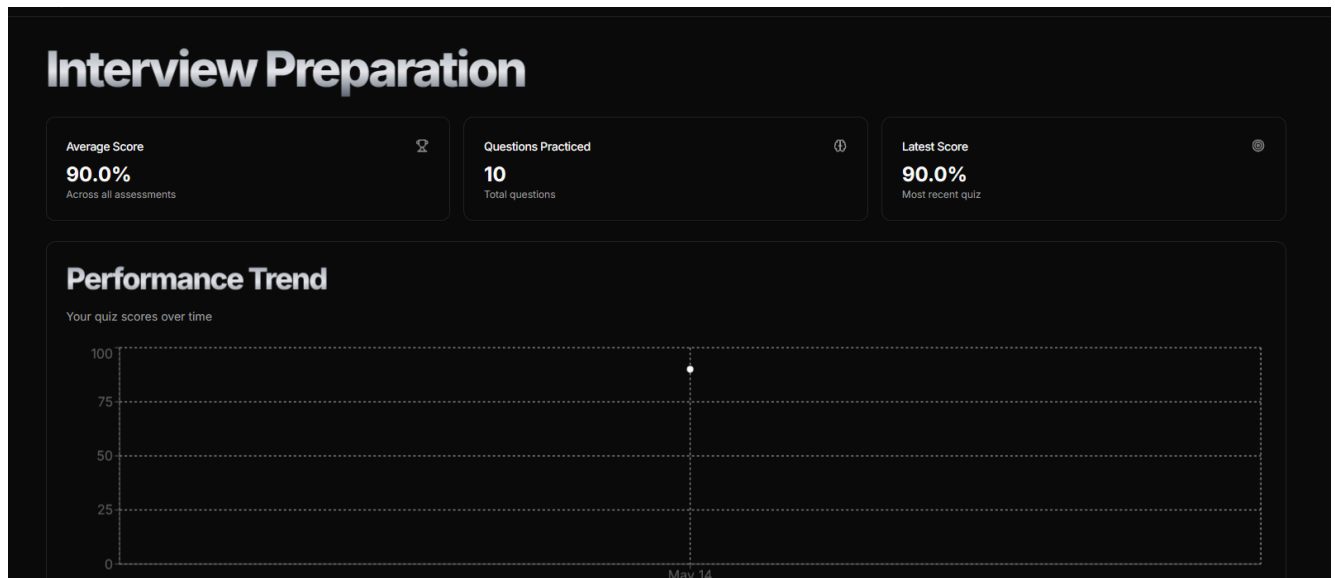
The **CoverLetterList** component presents saved cover letters in a clean, card-based layout. Each card highlights key details such as job title, company name, date of creation, and a brief job description, making it easy for users to browse their entries.

It includes interactive buttons to **view** or **delete** letters. Viewing redirects to a detailed page, while deletion is protected by a confirmation dialog to prevent accidental loss. Empty states are handled gracefully with prompts encouraging users to create their first cover letter.

Toast notifications provide real-time feedback for successful actions or errors, ensuring a smooth and user-friendly experience.



7.10 Interview Preparation – Practice Smart, Succeed Fast



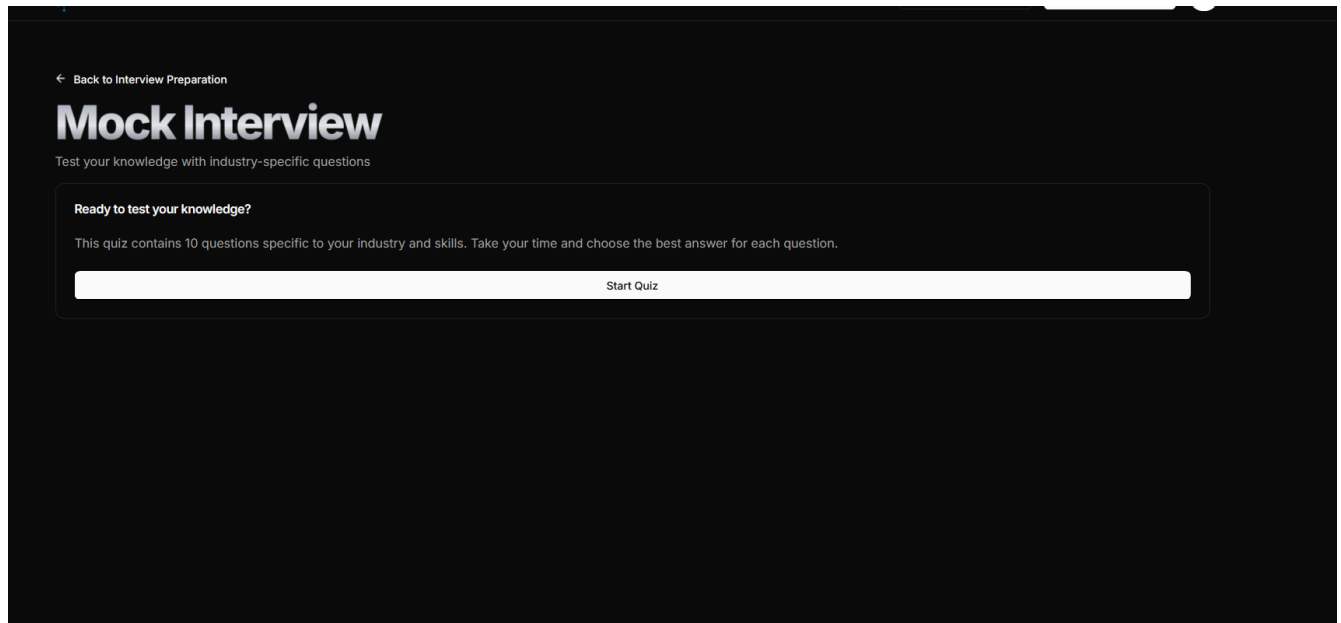
The **InterviewPrepPage** provides a structured and interactive space for users to prepare effectively for interviews. It loads detailed assessment data using the **getAssessments** action and distributes it across three key components.

- **StatsCards** presents a quick overview of important metrics like completed quizzes, average scores, and improvement rates—helping users instantly gauge their preparation status.
- **PerformanceChart** visualizes progress over time through graphs, allowing users to identify trends, strengths, and areas needing improvement.

This combination of summarized stats and visual analytics ensures users receive meaningful, data-driven insights to guide focused and efficient interview preparation.



7.11 Mock Interview –Your AI Powered Practice Ground



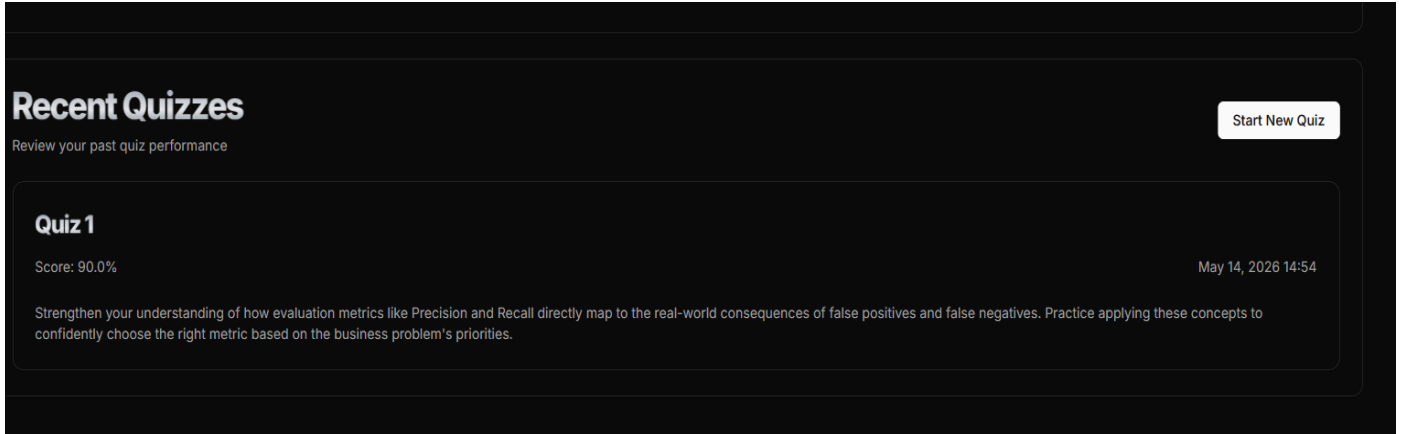
The **MockInterviewPage** provides a focused environment where users can simulate real-world interview scenarios to test their readiness. A clearly styled navigation button with a left arrow allows users to return smoothly to the Interview Preparation page.

The page features a bold “**Mock Interview**” title with gradient styling, followed by a brief description that sets the context. At its core, the **Quiz component** dynamically presents industry-specific questions, promoting active engagement and reinforcing learning through practice.

The layout is clean, responsive, and distraction-free, guiding users intuitively through the content. This page plays a crucial role in helping users identify strengths, target weak areas, and build confidence for real interviews.



7.12 Recent Quizzes



The **QuizList** component acts as a personal dashboard, showcasing a user’s previous mock interview attempts with clear performance summaries and review options.

At the top, a header titled “**Recent Quizzes**” is paired with a “**Start New Quiz**” button, encouraging continuous improvement. Each quiz is displayed as a clickable card showing the quiz number, score (to one decimal), attempt date (formatted via **date-fns**), and any improvement tips.

Clicking a card opens a **modal dialog** with detailed results via the **QuizResult** component. The modal is scrollable and responsive, allowing users to view insights without leaving the page.

Built with **useState** for tracking selections and **Next.js useRouter** for navigation, this component promotes reflection and growth by blending performance feedback with an intuitive user experience.

7.13 Resume Builder

The **EntryForm** component enables users to add and manage resume entries like experience, education, or projects through a dynamic and user-friendly interface. It uses **React Hook Form** for efficient input handling and **Zod** for real-time validation, with feedback provided via toast notifications.

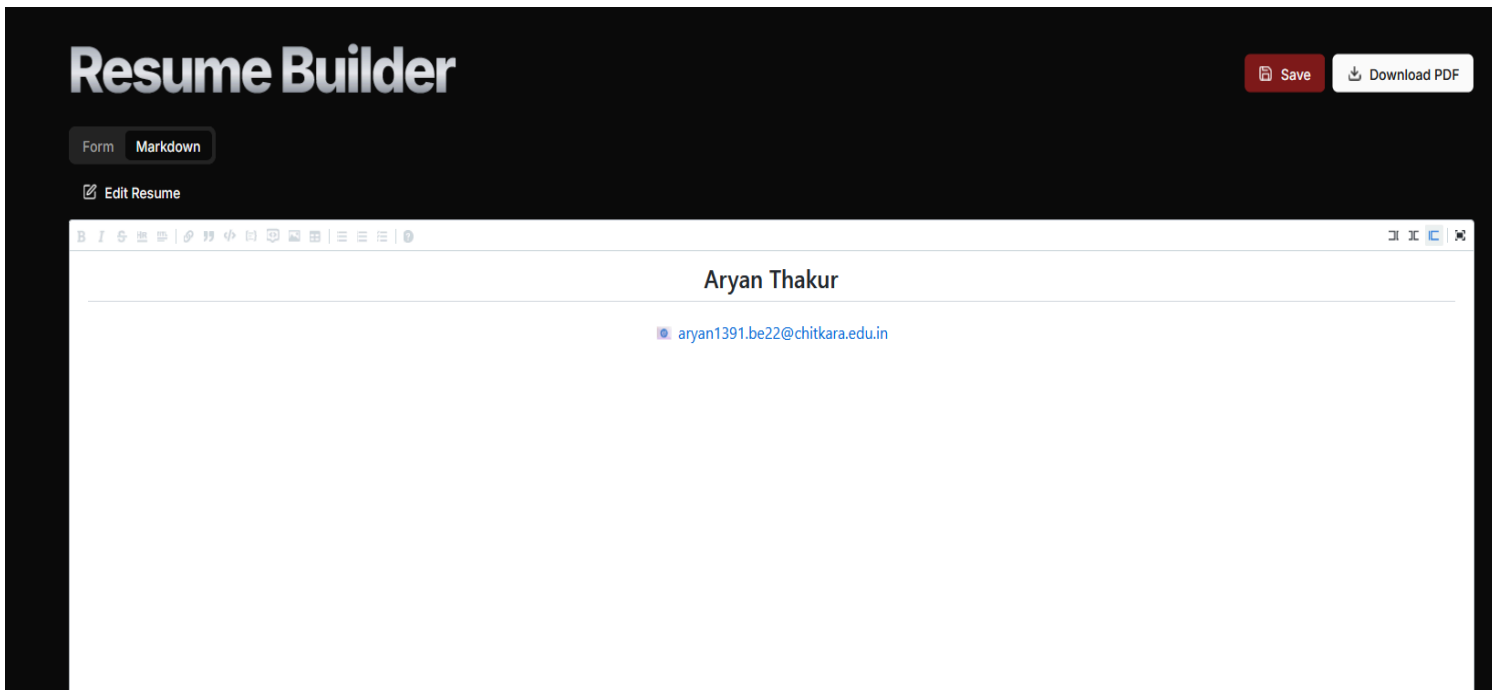
Each entry is shown as a styled card displaying the title, organization, duration, and description. Users can **add new entries**, delete existing ones, and mark roles as “current,” which auto-disables the end date field.

A standout feature is “**Improve with AI**”, which enhances entry descriptions using the **improveWithAI** action via a custom **useFetch** hook. The improved text is instantly populated, making content refinement seamless.

Dates are formatted using **date-fns** (e.g., "Jan 2023"), and form state is managed cleanly for error-free submissions. Overall, EntryForm makes resume editing intuitive, responsive, and AI-assisted—greatly improving both efficiency and content quality.



7.14 ATS Resume Builder



The **Resume Builder** component allows users to create, preview, and export professional resumes through a dynamic, React-based interface. Using **react-hook-form** with **Zod**, it ensures structured data entry and validation.

The builder offers two main views: a **form view** for structured input and a **markdown editor** for flexible customization. Contact info is auto-filled using **Clerk authentication**, with real-time updates reflected in the markdown preview.

It supports dynamic sections—experience, education, and projects—via the reusable **EntryForm** component. Users can switch between preview and edit modes with warnings to prevent unsaved changes.

Resumes can be exported as PDFs using **html2pdf.js**, and saved to the backend using a **saveResume** action wrapped in a **useFetch** hook. The UI, styled with **TailwindCSS** and **ShadCN**, provides a modern and responsive experience, enhanced by **Sonner** toast notifications for feedback.

Overall, the Resume Builder offers a seamless, customizable, and interactive solution for building resumes on the web.

7.15 Empowered with AI

ELEVATE AI

Create Cover Letter

Generate a tailored cover letter for your job application

Job Details
Provide information about the position you're applying for

Company Name
Microsoft

Job Title
Web Developer

Job Description
Developed and deployed five Google Chrome extensions, leveraging JavaScript, HTML, and CSS, resulting in a 20% increase in user engagement and a 15% reduction in support tickets within the first quarter of launch. Successfully integrated these extensions with existing Google Workspace applications, enhancing workflow efficiency and improving user experience via streamlined data access and automation features.

Generate Cover Letter

The **Improve with AI** button enhances user-written content (typically a description) using generative AI. Styled as a small, **ghost-type button**, it maintains a clean UI while offering powerful functionality.

The button triggers a `handleImproveDescription` function when clicked, which sends the current content to an AI service for enhancement. It remains disabled if the improvement is in progress (`isImproving` is true) or if the field is empty (`!watch("description")`), ensuring valid usage.

During processing, a **loading spinner** with the label “**Improving...**” is shown. Otherwise, a **sparkle icon** and “**Improve with AI**” label invite user interaction. The component integrates with **react-hook-form**, using `watch()` to monitor the input and `setValue()` to update it with AI-generated content.

This responsive and intuitive button elevates user-generated content while maintaining a smooth and guided experience.

Some Snippets From Server Side logics

7.16 Server-side Logic for AI-Powered Quiz Generation

```
import { db } from "@lib/prisma";
import { auth } from "bcryptjs/server";
import { GoogleGenerativeAI } from "@google/generative-ai";

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);
const model = genAI.getGenerativeModel({ model: "gemini-1.5-flash" });

export async function generateQuiz() {
  const { userId } = await auth();
  if (!userId) throw new Error("User not found");

  const user = await db.user.findUnique({
    where: {
      clerkUserId: userId,
    },
  });
  if (!user) throw new Error("User not found");

  try {
    const prompt = `
    Generate 10 technical interview questions for a ${
      user.industry
    } professional with
    user.skills?.length > 0 ? 'with expertise in ${user.skills.join(", ")}' : ""
    `;

    Each question should be multiple choice with 4 options.

    Return the response in this JSON format only, no additional text:
    {
      "questions": [
        {
          "question": "string",
          "options": ["string", "string", "string", "string"],
          "correctAnswer": "string",
          "explanation": "string"
        }
      ]
    };

    const result = await model.generateContent(prompt);
    const response = result.response;
    const text = response.text();
    const cleanedText = text.replace(/```json/g, "").trim();
    const quiz = JSON.parse(cleanedText);

    return quiz.questions;
  } catch (error) {
    console.log("Error generating quiz:", error);
    throw new Error("Failed to generate quiz");
  }
}
```

In order to provide a dynamic and tailored learning experience, the application incorporates Google's Generative AI (Gemini) into its backend architecture. This integration enables the automatic generation of technical interview questions specific to a user's industry domain and individual skillset.

Upon successful user authentication, the backend retrieves relevant user details such as industry type and skills from the database. This information is then programmatically embedded into a carefully constructed prompt, which is passed to the Generative AI API. The AI processes this context and returns 10 multiple-choice questions, each consisting of four options, a correct answer, and a brief explanation — all in a clean JSON format.

This eliminates the need for a static question bank and ensures that each user receives a unique, relevant, and adaptive quiz. This functionality not only enhances personalization but also demonstrates the power of AI in real-time educational content generation.

This sub-module stands as a perfect example of AI-augmented backend logic that seamlessly merges user data with generative capabilities to boost engagement and provide intelligent learning outcomes.

7.17 Server-side Logic for AI-Powered Quiz Feedback

```
export async function saveQuizResult({ questions, answers, score }) {
  const { userId } = await auth();
  if (!userId) throw new Error("User not found");

  const user = await db.user.findUnique({
    where: {
      clerkUserId: userId,
    },
  });
  if (!user) throw new Error("User not found");

  const questionResults = questions.map((q, index) => ({
    question: q.question,
    answer: q.correctAnswer,
    userAnswer: answers[index],
    isCorrect: q.correctAnswer === answers[index],
    explanation: q.explanation,
  }));

  const wrongAnswers = questionResults.filter((q) => !q.isCorrect);
  let improvementTip = null;

  if (wrongAnswers.length > 0) {
    const wrongQuestionsText = wrongAnswers
      .map(
        (q) =>
        `Question: "${q.question}"\nCorrect Answer: "${q.correctAnswer}"\nUser Answer: "${q.userAnswer}"`
      )
      .join("\n\n");

    const improvementPrompt = `
    The user got the following ${user.industry} technical interview questions wrong:
    ${wrongQuestionsText}

    Based on these mistakes, provide a concise, specific improvement tip.
    Focus on the knowledge gaps revealed by these wrong answers.
    Keep the response under 2 sentences and make it encouraging.
    Don't explicitly mention the mistakes, instead focus on what to learn/practice.
    `;

    try {
      const result = await model.generateContent(improvementPrompt);
      const response = result.response;
      improvementTip = response.text().trim();
    } catch (error) {
      console.log("Error generating improvement tip:", error);
    }
  }

  try {
    console.log("QuestionResults", questionResults);

    const assessment = await db.assessment.create({
      data: {
        userId: user.id,
        quizScore: score,
        category: "technical",
        improvementTip,
        questions: questionResults, // 🟢 THIS is the fix
      },
    });
  } catch (error) {
    console.log("Error saving assessment:", error);
  }

  return assessment;
}
```

The figure above displays the backend implementation of the **saveQuizResult** function, which handles quiz submission and stores assessment data in the database. Written in **TypeScript** and running on the server side, this function processes quiz responses, checks correctness, and logs detailed results for each question.

If incorrect answers are detected, it dynamically constructs a **custom prompt** using the user's industry context and question data. This prompt is then sent to an **AI model** to generate a short, encouraging improvement tip. The final output, along with the quiz details, is saved to the **Assessment** table using Prisma.

This logic enables personalized learning by providing targeted feedback, improving user performance through intelligent and adaptive responses.

7.18 Server-side Logic for AI-Generated Industry Insights

```

83
84 import { db } from "@lib/prisma";
85 import { auth } from "@clerk/nextjs/server";
86 import { GoogleGenerativeAI } from "@google/generative-ai";
87
88 // Initialize Gemini AI
89 const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);
90 const model = genAI.getGenerativeModel({ model: "gemini-1.5-flash" });
91
92 export const generateAIInsights = async (industry) => {
93   const safeIndustry = industry || "null-industry";
94
95   const prompt = `
96     Analyze the current state of the ${safeIndustry} industry and provide insights in ONLY the following JSON format without any additional notes or explanations:
97     {
98       "salaryRanges": [
99         { "role": "string", "min": number, "max": number, "median": number, "location": "string" }
100       ],
101       "growthRate": number,
102       "demandLevel": "High" | "Medium" | "Low",
103       "topSkills": ["skill1", "skill2", "skill3", "skill4", "skill5"],
104       "marketOutlook": "Positive" | "Neutral" | "Negative",
105       "keyTrends": ["trend1", "trend2", "trend3", "trend4", "trend5"],
106       "recommendedSkills": ["skill1", "skill2", "skill3", "skill4", "skill5"]
107     }
108
109     IMPORTANT: Return ONLY the JSON. No additional text, notes, or markdown formatting.
110   `;
111
112   const result = await model.generateContent(prompt);
113   const response = result.response;
114   const text = await response.text();
115   const cleanedText = text.replace(/```(?:json)?\n?/g, "").trim();
116
117   let insights;
118   try {
119     insights = JSON.parse(cleanedText);
120   } catch (error) {
121     console.error("✖ JSON parsing failed:", error);
122     return null;
123   }
124
125   // Fix ENUM mapping for Prisma
126   const demandLevelMap = { High: "HIGH", Medium: "MEDIUM", Low: "LOW" };

```

The figure above shows the server-side function `generateAIInsights`, which uses Google Generative AI (Gemini) to generate structured industry insights based on a given input. This function is part of the backend for the Industry Insights feature of the AI Career Coach platform.

The function builds a strict prompt requesting insights in a predefined JSON format, covering data points like:

- Salary ranges
- Industry growth rate
- Skill demand level
- Market outlook
- Top and recommended skills
- Key trends

The AI response is parsed, cleaned, and converted into a usable JavaScript object. Error handling ensures graceful failure if JSON formatting is incorrect. This structured output is later stored and used to render charts and insights on the user dashboard.

By automating data generation through AI, this logic ensures real-time, industry-specific updates that are consistent, reliable, and tailored for each user's career path.

7.19 Server-side Logic for Fetching and Formatting Industry Insights

```
// Fix ENUM mapping for Prisma
const demandLevelMap = { High: "HIGH", Medium: "MEDIUM", Low: "LOW" };
const marketOutlookMap = {
  Positive: "POSITIVE",
  Neutral: "NEUTRAL",
  Negative: "NEGATIVE",
};

insights.demandLevel = demandLevelMap[insights.demandLevel] || "MEDIUM";
insights.marketOutlook =
  marketOutlookMap[insights.marketOutlook] || "NEUTRAL";

return insights;
};

export async function getIndustryInsights() {
  const { userId } = await auth();
  if (!userId) throw new Error("Unauthorized");

  const user = await db.user.findUnique({
    where: { clerkUserId: userId },
    include: {
      industryInsight: true,
    },
  });

  if (!user) throw new Error("User not found");
}
```

The figure above illustrates the backend logic responsible for retrieving and formatting AI-generated industry insights for each user. It includes two main functions:

- **Enum Mapping Fixes:** The first section ensures compatibility between AI-generated values and the database schema by mapping raw **demandLevel** and **marketOutlook** strings into strict enums accepted by Prisma ORM. Defaults like **"MEDIUM"** and **"NEUTRAL"** are applied when needed.
- **getIndustryInsights Function:** This function securely fetches the logged-in user's data using **auth()** and retrieves their corresponding **industryInsight** record from the database. It uses **clerkUserId** to uniquely identify users and includes related industry insight data through Prisma's **include** feature.

This logic enables secure, structured, and correctly formatted retrieval of AI-driven insights, supporting personalized dashboards and recommendations across the platform.

7.20 Server-side Logic for Cover-Letter Generation Module

```
import { db } from "@lib/prisma";
import { auth } from "@clerk/nextjs/server";
import { GoogleGenerativeAI } from "@google/generative-ai";

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);
const model = genAI.getGenerativeModel({ model: "gemini-1.5-flash" });

export async function generateCoverLetter(data) {
  const { userId } = await auth();
  if (!userId) throw new Error("Unauthorized");

  const user = await db.user.findUnique({
    where: { clerkUserId: userId },
  });
  if (!user) throw new Error("User not found");

  const prompt = `
  Write a professional cover letter for a ${data.jobTitle} position at ${data.companyName}.

  About the candidate:
  - Industry: ${user.industry}
  - Years of Experience: ${user.experience}
  - Skills: ${user.skills?.join(", ")}
  - Professional Background: ${user.bio}

  Job Description:
  ${data.jobDescription}

  Requirements:
  1. Use a professional, enthusiastic tone
  2. Highlight relevant skills and experience
  3. Show understanding of the company's needs
  4. Keep it concise (max 400 words)
  5. Use proper business letter formatting in markdown
  6. Include specific examples of achievements
  7. Relate candidate's background to job requirements

  Format the letter in markdown.
  `;

  try {
    const result = await model.generateContent(prompt);
    const content = result.response.text().trim();

    const coverLetter = await db.coverLetter.create({
      data: {
        content,
        jobDescription: data.jobDescription,
        companyName: data.companyName,
        jobTitle: data.jobTitle,
        status: "completed",
        userId: user.id,
      },
    });

    return coverLetter;
  } catch (error) {
    console.error("Error generating cover letter:", error.message);
    throw new Error("Failed to generate cover letter");
  }
}
```

This module handles the creation, retrieval, and deletion of AI-generated cover letters tailored to specific job applications. It integrates user authentication, database storage, and Google's Gemini AI model to streamline the process of writing professional cover letters.

✓ Features

- **Generate AI-powered cover letters** based on user profile and job details.
- **Authenticate users** securely using Clerk.
- **Store and manage cover letters** using Prisma with a PostgreSQL or other supported database.
- **Fetch cover letters** in descending order of creation time.
- **Delete cover letters** belonging to the authenticated user.

🔒 Authentication

Every function validates the user via Clerk's server-side `auth()` method. Unauthorized users are denied access.

🤖 AI Integration

- Uses **Google Generative AI (Gemini 1.5 Flash)** for generating personalized, professional-quality cover letters.
- Prompt includes user industry, experience, skills, and a detailed job description.

7.21 Server-side Logic Technical Quiz & Assessment Module

```
const user = await db.user.findUnique({
  where: { clerkUserId: userId },
  select: {
    industry: true,
    skills: true,
  },
});

if (!user) throw new Error("User not found");

const prompt = `
Generate 10 technical interview questions for a ${
  user.industry
} professional${
  user.skills?.length ? ` with expertise in ${user.skills.join(", ")}` : ""
}.

Each question should be multiple choice with 4 options.

Return the response in this JSON format only, no additional text:
{
  "questions": [
    {
      "question": "string",
      "options": ["string", "string", "string", "string"],
      "correctAnswer": "string",
      "explanation": "string"
    }
  ]
}
`;
```

This module provides functionality to generate personalized technical quizzes, evaluate user performance, and store quiz assessments. It leverages Google Gemini AI to create questions and improvement tips, while securely handling user data and results with Clerk and Prisma.

✓ Features

- **AI-powered quiz generation** based on the user's industry and skills.
- **Automatic result evaluation**, including scoring and feedback.
- **Smart improvement suggestions** if the user answers incorrectly.
- **Secure data handling** using Clerk for authentication and Prisma for persistence.
- **Assessment history tracking** per user.

🤖 AI Integration

- Uses gemini-1.5-flash from Google Generative AI to:
 - Generate 10 multiple-choice questions per quiz.
 - Provide detailed explanations for each answer.
 - Generate personalized improvement tips if any questions are answered incorrectly.

🔑 Authentication

- Uses @clerk/nextjs/server to authenticate users server-side.
- Each operation checks user identity before allowing access to quiz generation or assessment data.

7.22 Server-side Logic for Resume Management & AI Improvement Module

```
export async function improveWithAI({ current, type }) {
  const { userId } = await auth();
  if (!userId) throw new Error("Unauthorized");

  const user = await db.user.findUnique({
    where: { clerkUserId: userId },
    include: {
      industryInsight: true,
    },
  });

  if (!user) throw new Error("User not found");

  const prompt = `
  As an expert resume writer, improve the following ${type} description for a ${user.industry} professional.
  Make it more impactful, quantifiable, and aligned with industry standards.
  Current content: "${current}"

  Requirements:
  1. Use action verbs
  2. Include metrics and results where possible
  3. Highlight relevant technical skills
  4. Keep it concise but detailed
  5. Focus on achievements over responsibilities
  6. Use industry-specific keywords

  Format the response as a single paragraph without any additional text or explanations.
  `;
}
```

This module enables users to **save**, **retrieve**, and **enhance** their resumes using AI. It leverages Clerk for authentication, Prisma for persistence, and Google Gemini AI to optimize resume content with industry best practices.

✓ Features

- **Save or update** resumes using the upsert pattern.
- **Fetch the latest resume** tied to the authenticated user.
- **AI-powered content improvement** for specific resume sections (e.g., experience, summary).
- **Cache revalidation** for up-to-date frontend rendering after saving.

🔒 Authentication

Every operation checks for valid user sessions using `@clerk/nextjs/server`. Users must be authenticated to interact with resume data.

🤖 AI Enhancement Logic

Powered by `gemini-1.5-flash` from Google Generative AI, the enhancement prompt ensures:

- Impactful phrasing with **action verbs**
- Results-oriented writing using **metrics and achievements**
- Use of **industry-specific keywords** for better ATS compatibility

7.23 Server-side Logic for User Profile & Industry Insight Management

```
export async function updateUser(data) {
  console.log("Received data:", data);

  const { userId } = await auth();
  if (!userId) throw new Error("unauthorized");

  const user = await db.user.findUnique({
    where: {
      clerkUserId: userId,
    },
  });
  if (!user) throw new Error("User not found");

  try {
    const result = await db.$transaction(
      async (tx) => {
        // Check if industry insight exists
        let industryInsight = await tx.industryInsight.findUnique({
          where: {
            industry: data.industry,
          },
        });

        // If not, create a default one (AI-generated values can be added later)
        if (!industryInsight) {
          const insights = await generateAIInsights(data.industry);
          industryInsight = await db.industryInsight.create({
            data: {
              industry: data.industry,
            },
          });
        }
      }
    );
  } catch (error) {
    console.error("Transaction failed:", error);
    throw error;
  }
}
```

This module handles **user profile updates**, manages **industry-specific insights**, and checks the **onboarding status** of authenticated users. It ensures a tight integration between the user's data and AI-generated industry trends to personalize the user experience.



Features

- Updates and persists user profile information (industry, experience, bio, skills).
- Ensures **industryInsight** data exists for the user's chosen domain.
- Checks if a user has completed the onboarding process.



Authentication

All functions utilize `@clerk/nextjs/server` to validate session-based authentication before performing any operations.

⚙️ Server Functions

All methods are executed securely on the server using `use server` directive.



Database Involvement

- **Models Used:** User, IndustryInsight
- **Key Fields:**
 - User.industry, experience, bio, skills
 - IndustryInsight.industry, demandLevel, marketOutlook, etc.

⚠️ Error Handling

- Throws detailed errors for unauthorized access, user not found, or transactional failures.
- Logs critical issues to server logs for easier debugging.

8. Testing & Validation

8.1 API Testing

All API endpoints were tested using Postman. We verified:

- User registration and login with JWT token issuance
- Resume submission and retrieval
- Interview question generation
- Dashboard data retrieval and

updates Sample test case:

- **Endpoint:** `/api/auth/login`
- **Method:** POST
- **Status Code:** 200 OK
- **Response:** JWT token and user details

8.2 UI Testing

UI components were tested manually across Chrome, Firefox, and Edge for:

- Responsive design behavior
- Button click interactions
- Form validation
- Notification handling

Console logs were reviewed to identify and fix any JavaScript or rendering errors.

8.3 Integration Testing

We tested the complete flow:

- From user signup → resume generation → interview simulation → dashboard insights.
- Ensured that database entries matched front-end updates and responses.

8.4 Error Handling

- Simulated API failures and tested fallback messages.
- Handled edge cases such as empty form submissions, invalid inputs, and server timeout gracefully

9. Security & Performance

1 HTTPS & Secure Communication

All API requests are made over HTTPS to ensure secure transmission of sensitive data such as login credentials, personal information, and resume content.

2 JWT Authentication & Session Management

The system uses JSON Web Tokens (JWT) for secure, stateless authentication. Clerk.dev is integrated to manage user sessions and ensure identity verification. JWT expiry and token refresh policies are enforced.

3 Performance Optimization

- Database queries optimized using Prisma ORM features like indexes and relations.
- Caching strategies considered for frequently accessed data.
- Lazy loading and code splitting implemented on the frontend.

4 Security Best Practices

- Passwords and tokens never stored in plain text.
- Input validation on all forms to prevent injection attacks.
- Dependencies scanned for vulnerabilities using npm audit.

These security and performance measures collectively help maintain a safe, reliable, and scalable platform.

10. Limitations

- **Internet Dependency:** The application relies on a stable internet connection for users to access AI features, update profiles, and receive personalized recommendations. Limited or no connectivity may disrupt the user experience.
- **Security Risks:** Despite using JWT for authentication and secure data handling, the system can still be exposed to cyber threats such as phishing, unauthorized access, or data breaches, especially if best practices are not consistently maintained.
- **AI Accuracy and Bias:** The AI-driven recommendations depend on the quality and fairness of underlying data. Inaccurate suggestions or biased responses may misguide users if not properly monitored and fine-tuned.
- **Scalability Concerns:** As the number of users and AI interactions grows, server performance may degrade without appropriate scaling and optimization, particularly during peak usage times.
- **Limited Offline Access:** Users must be online to interact with the platform, which restricts access for individuals in low-connectivity areas or those preferring offline career guidance.

11. Challenges Faced

During the development of the AI Career Coach platform, the team encountered several technical and non-technical challenges that tested our problem-solving and project management skills. These challenges provided valuable learning experiences and shaped the final design of the application:

1. Integration of AI APIs:

Incorporating Gemini APIs for personalized career guidance and resume analysis required careful handling of prompts, data flow, and asynchronous operations. Achieving accurate and unbiased recommendations demanded multiple iterations and refinements.

2. Frontend-Backend Synchronization:

Ensuring smooth communication between the frontend (Next.js) and backend (Express.js) posed challenges, especially in managing API calls, securing data transmission using JWT, and handling dynamic user input during resume building and interview simulations.

3. Database Schema Complexity:

Designing an efficient and scalable relational schema in PostgreSQL that supports multiple entities—users, resumes, job listings, and interview records—was complex. Maintaining referential integrity and optimizing query performance required the use of Prisma ORM and extensive debugging.

4. Authentication and Security:

Implementing JWT-based authentication and role-based access control involved handling sensitive user data securely. Ensuring protection against common web vulnerabilities (e.g., XSS, CSRF) demanded consistent security practices and dependency management.

12. Future Scope

- **Enhanced Security:** Introduce multi-factor authentication, end-to-end encryption, and regular security audits to strengthen system protection and ensure data integrity.
- **Offline Booking Capabilities:** Implement SMS-based ride requests or call center support to accommodate users with limited internet access or those preferring traditional booking methods.
- **Scalability Improvements:** Utilize load balancing and distributed databases to efficiently manage increased user traffic and data volumes, ensuring system performance at scale.
- **User Accessibility:** Add multilingual support and accessibility features to make the platform inclusive for users with diverse needs, including those with disabilities.
- **Advanced Analytics:** Integrate robust analytics tools to track ride demand, monitor driver performance, and analyze user engagement for strategic decision-making.
- **Blockchain Integration:** Explore blockchain to secure transaction data, ensuring tamper-proof ride records and transparent payment histories.
- **Mobile App Development:** Build a dedicated mobile app to enhance convenience and usability for users on smartphones and tablets.

13. Conclusion

The AI Career Coach application, developed using modern technologies such as Next.js, Node.js, Express, and PostgreSQL, offers a comprehensive and intelligent solution for career planning and development. By integrating AI-driven features through Gemini APIs, the platform provides personalized guidance, resume and cover letter generation, mock interview simulations, and career path recommendations tailored to individual users. The use of Material UI ensures a responsive and user-friendly frontend, while secure authentication with JWT and real-time API interactions strengthen the backend's reliability and performance.

The application supports users in exploring job roles aligned with their skills, tracking their progress, and accessing updated industry insights, ultimately enhancing career readiness. While the system brings significant benefits, it is not without limitations, such as dependency on internet access, potential scalability issues, and the need for continuous AI model improvements to avoid biased or inaccurate suggestions.

Despite these challenges, the AI Career Coach stands as a forward-thinking platform in the EdTech and career guidance space. Future developments could include AI-powered chat assistants, deeper labor market analytics, and support for multilingual users, further broadening accessibility and impact. Overall, the project highlights the power of combining AI with full-stack web development to empower users on their professional journey.



14. Bibliography/References

- To develop and integrate advanced features in AI Carrier Coach , we referred to the following technologies, tools, and resources:
- **Authentication & Security**
 - 🔑 **JWT Authentication** – jwt.io
 - 🛡️ **Clerk Authentication (User Management)** – www.clerk.dev
- **Backend & Database**
 - 🔧 **Node.js & Express.js for API development** – nodejs.org | expressjs.com
 - 🗄️ **PostgreSQL & Supabase for database storage** – www.postgresql.org | <https://supabase.com/>
 - 🔧 **Prisma ORM for database management** – www.prisma.io
- **AI & Industry Insights**
 - 🤖 **Gemini APIs for AI-powered career analysis** – ai.google.dev
 - 📊 **Industry Trends & Insights** – [LinkedIn Economic Graph](#) | Glassdoor Research
- **Frontend Development**
 - 💻 **Next.js Documentation** – nextjs.org
 - 🎨 **Chakra UI for UI Components** – chakra-ui.com
 - 🧩 **shadcn/ui (Accessible, Themeable Components)** – ui.shadcn.dev
- **Testing & Monitoring**
 - 🔧 **Postman for API testing & middleware validation** – www.postman.com
 - 📈 **Log Monitoring & Performance Analysis** – [Datadog](https://datadog.com) | [Sentry](https://sentry.io)
- **Bonus References**
 - 🛡️ **OWASP (Web Security Best Practices)** – owasp.org
 - ⚙️ **Render (Full-stack App Deployment)** – render.com
 - 📖 **DevDocs (Aggregated Dev Documentation)** – devdocs.io

